

Análisis de Modelos de Aprendizaje Automático para Clasificación de Lesiones Pre-Tumorales

Miguel Ángel Ávila Carrero, Carolina Kung, Micaela Pego, Malena Sanz

1. Separación de datos

En todo problema de aprendizaje automático, es fundamental conocer y explorar los datos disponibles antes de entrenar modelos. Algunas de las preguntas clave en esta etapa son: ¿cuántas instancias tenemos?, ¿cómo están distribuidas las clases?, ¿cómo deberíamos dividir los datos para entrenar y validar los modelos?

En nuestro caso, contamos con un dataset de 500 muestras que contienen mediciones de ARN de 200 genes, extraídas de pacientes con lesiones pre-tumorales. Cada muestra está etiquetada como buen pronóstico ($\text{target} = 1$) o mal pronóstico ($\text{target} = 0$).

En primer lugar, decidimos dividir el dataset en dos partes:

- Un conjunto de desarrollo (90 % de las instancias), que se utilizará para entrenamiento y validación.
- Un conjunto de control (10 % de las instancias), reservado exclusivamente para la evaluación final del desempeño del modelo.

Durante la exploración inicial del dataset, observamos que las clases no están balanceadas: menos del 30 % de las muestras tienen la etiqueta positiva ($\text{target} = 1$). Además, dado que la cantidad total de datos es limitada (sólo 500 muestras), decidimos implementar una estrategia de separación que preserve la proporción de clases en ambos conjuntos, para asegurar que en el conjunto de control existan suficientes etiquetas positivas.

Para ello, construimos una función llamada `separa_df()`, que selecciona al azar un 10 % de las muestras de cada clase utilizando una distribución uniforme. Esto asegura que ambas clases estén representadas proporcionalmente en el conjunto de desarrollo y el de control.

Las proporciones de etiquetas positivas quedaron de la siguiente manera:

- Proporción en el dataset original: 0.294
- Proporción en el conjunto de desarrollo: 0.295
- Proporción en el conjunto de control: 0.286

Esto confirma que la separación fue efectiva para preservar de manera aproximada la distribución original de clases, lo cual es clave para realizar una evaluación con los datos de control.

2. Construcción de modelos

En este ejercicio construimos y evaluamos modelos de tipo árbol de decisión. El objetivo fue entrenar un modelo con hiperparámetros predefinidos y estimar de manera realista su desempeño mediante validación cruzada, aprovechando así al máximo la cantidad limitada de datos disponibles.

En primer lugar, entrenamos un árbol de decisión con hiperparámetros por defecto y con altura máxima igual a 3. Para evaluar su desempeño utilizamos validación cruzada con cinco folds (K=5). Dado el desbalance existente entre las clases, fue necesario aplicar folds estratificados de modo tal que cada partición conserve la misma proporción de la clase positiva y negativa que presenta el conjunto original. De esta forma, evitamos que alguno de los folds quede sin ejemplos de la clase minoritaria, lo cual afectaría negativamente la estimación de las métricas. Cabe mencionar que algunas funciones de sklearn hacen folds estratificados por defecto.

Las métricas de interés eran Accuracy, el área bajo la curva Precision-Recall (AUPRC) y el área bajo la curva ROC (AUCROC). En todos los casos estas métricas se calcularon en el conjunto de entrenamiento y en el de validación de cada fold, y posteriormente se reportaron sus promedios. Es importante destacar que para AUPRC y AUCROC se utilizaron las probabilidades de pertenencia a la clase positiva entregadas por el modelo (y no las etiquetas duras).

Además de los promedios, se calculó el score global. Para construirlo se concatenaron las predicciones de validación de los cinco folds, de manera que cada instancia del dataset fue evaluada por un modelo que no la había visto durante su entrenamiento. De este modo se obtuvo un único vector de predicciones de tamaño igual al del dataset original, con el cual se calcularon nuevamente Accuracy, AUPRC y AUCROC.

Se implementaron dos formas de llevar adelante este procedimiento. En la primera, más “manual”, generamos folds estratificados usando la función **StratifiedKFold**, posteriormente separando el dataset de desarrollo en los 5 subconjuntos sobre los cuales entrenaremos un árbol para cada uno de los k-1 sets de entrenamiento, guardando cada una de las predicciones sobre un fold de validación. Para el score de entrenamiento, tomamos los k-1 folds y los concatenamos, guardando las predicciones sobre los mismos datos que se entrenó. Los resultados correspondientes se encuentran en el Cuadro 1.

Permutación	Accuracy (Training)	Accuracy (Validation)	AUPRC (Training)	AUPRC (Validation)	AUROC (Training)	AUROC (Validation)
1	0.831	0.714	0.732	0.450	0.863	0.682
2	0.845	0.733	0.681	0.486	0.834	0.745
3	0.850	0.622	0.744	0.380	0.855	0.617
4	0.823	0.778	0.691	0.600	0.822	0.828
5	0.828	0.756	0.698	0.574	0.818	0.758
Promedio	0.835	0.721	0.709	0.498	0.838	0.726
Global	NaN	0.721	NaN	0.486	NaN	0.722

Cuadro 1: Resultados de las múltiples iteraciones del cross validate manual, usando StratifiedKFolds. Se anexa el promedio de estas iteraciones para training y validation; como tambien el promedio global de validacion

En la segunda forma, más automatizada, se emplearon las funciones **cross_validate** y **cross_val_predict** de Scikit-learn. Con **cross_validate** se calcularon los scores de entrenamiento y validación de cada fold, mientras que con **cross_val_predict** se obtuvieron las predicciones necesarias para calcular el score global. Los resultados se hallan en el Cuadro 2.

Permutación	Accuracy (Training)	Accuracy (Validation)	AUPRC (Training)	AUPRC (Validation)	AUROC (Training)	AUROC (Validation)
1	0.831	0.670	0.675	0.369	0.809	0.556
2	0.825	0.722	0.678	0.490	0.818	0.744
3	0.817	0.778	0.673	0.567	0.847	0.758
4	0.850	0.744	0.706	0.510	0.847	0.738
5	0.853	0.667	0.729	0.348	0.865	0.614
Promedio	0.835	0.716	0.692	0.457	0.837	0.682
Global	NaN	0.716	NaN	0.472	NaN	0.692

Cuadro 2: Resultados de las múltiples iteraciones del cross validate automatico, usando sklearn. Se anexa el promedio de estas iteraciones para training y validation; como tambien el promedio global de validacion

Observando los resultados presentados en los Cuadros 1 y 2, podemos notar que los scores son consistentemente mayores para los conjuntos de entrenamiento que para los de validación, tanto

fold a fold como en el promedio de cada métrica. Esto es un comportamiento esperable, ya que el modelo fue entrenado sobre los datos de entrenamiento y, por lo tanto, tiende a ajustarse mejor a esos ejemplos.

Al analizar las métricas, se ve que Accuracy presenta los valores más altos en comparación con AUPRC y AUC ROC. Si bien esto podría dar la impresión de que el modelo es muy bueno, debemos considerar que en datasets desbalanceados como el nuestro, Accuracy puede ser engañosa.

En contraste, AUPRC muestra valores más bajos, lo cual es razonable ya que esta métrica se enfoca principalmente en la clase positiva. Al tener un desbalance de clases significativo, usar una métrica que dependa del recall y precision no va a ser lo más conveniente.

Por otro lado, AUC ROC presenta valores entre Accuracy y AUPRC. Esta métrica evalúa la capacidad del modelo para discriminar entre clases positivas y negativas, considerando tanto verdaderos positivos como falsos positivos. Sabiendo que AUC ROC es menos sensible ante el desbalance de clases, tomamos esta métrica como un valor más realista de predicción.

Al comparar los promedios de los folds con los scores globales, se observa que la Accuracy global coincide con la Accuracy promedio, lo cual era esperable dado que para esta métrica los valores son exactamente iguales. En cambio, para AUPRC y AUC ROC, el score global en principio difiere del promedio de folds. A la hora de evaluar el desempeño de manera realista, confiamos más en el score global debido a que este permite obtener una predicción no tan afectada por las distintas proporciones dentro de los folds de validación, al unificarlos todos.

Finalmente, los resultados obtenidos mediante los dos enfoques implementados (manual en Cuadro 1 y usando funciones de Scikit-learn en Cuadro 2) no coinciden exactamente, aunque sí son similares. Creemos que se atribuye a como se formaron los folds para cada enfoque.

En segundo lugar, exploramos el impacto de modificar los hiperparámetros del árbol mediante un `ParameterGrid`, considerando distintas combinaciones de `criterion` (gini y entropy) y `max_depth` (3, 5 y sin límite). A diferencia de un `GridSearchCV`, aquí se buscó evaluar explícitamente cada combinación sin delegar el procedimiento a una búsqueda automática.

Criterio de corte	Altura máxima	Accuracy (Training)	Accuracy (Validation)
Gini	3	0.835	0.716
Gini	5	0.933	0.687
Gini	Infinito	1.000	0.672
Entropy	3	0.808	0.705
Entropy	5	0.914	0.685
Entropy	Infinito	1.000	0.679

Cuadro 3: Resultados de los diferentes criterios de corte y alturas máximas

En el Cuadro 3, se observa que aumentar la altura máxima del árbol mejora el accuracy en el conjunto de entrenamiento, pero al mismo tiempo lo reduce en validación. Esto tiene sentido: un árbol sin límite de profundidad puede aprender todas las decisiones necesarias para clasificar perfectamente los datos de entrenamiento, pero pierde capacidad de generalización a datos nuevos.

En cuanto al criterio de corte, su impacto en la Accuracy no parece tan grande. Por ejemplo, para árboles de altura máxima 3, Gini supera ligeramente a Entropy, mientras que para altura 5 la situación se invierte. Sin embargo, las diferencias son pequeñas y sería necesario hacer más pruebas para sacar conclusiones sólidas.

En resumen, este análisis muestra que limitar la profundidad del árbol es una forma efectiva de controlar la complejidad del modelo y reducir el riesgo de overfitting, logrando un mejor equilibrio entre ajuste y capacidad de generalización.

3. Selección de modelos

En este ejercicio, nos dedicamos a explorar distintas combinaciones de algoritmos de aprendizaje con diferentes configuraciones con el objetivo de encontrar el mejor modelo de cada familia. Para ello, realizamos una experimentación utilizando `RandomizedSearchCV`, que estima la métrica de performance AUCROC a través de validación cruzada con 5 folds, haciendo un número de iteraciones definido muestreando valores de un diccionario con los hiperparámetros más relevantes y los rangos que queríamos explorar. Decidimos el número de iteraciones en base a la cantidad de combinaciones posibles de hiperparámetros para cada algoritmo. En el caso de KNN y SVM, elegimos usar 50 iteraciones, mientras que para el árbol de decisiones, como exploramos más hiperparámetros, usamos 200.

3.1.

3.1.1. Árbol de decisión

Los parámetros que exploramos fueron:

- `max_depth`: `np.arange(2, 10)`

Este hiperparámetro establece la profundidad máxima del árbol y es clave para controlar la complejidad del modelo; un valor alto permite que el modelo capture patrones más complejos en los datos de entrenamiento, pero aumenta el riesgo de overfitting. Por otro lado, un valor bajo puede evitar que el modelo aprenda suficiente de los datos (underfitting).

Justificación del rango: Inicialmente, debido a la gran cantidad de atributos del dataset, probamos valores altos (entre 2 y 50). Sin embargo, notamos que el valor óptimo nunca superaba el valor 10. Por ello, en las iteraciones siguientes, acortamos el rango a entre 2 y 9, lo que consideramos un rango más razonable.

- `min_samples_split`: `np.arange(2, 50)`

Es el número mínimo de muestras requeridas para que un nodo interno pueda ser dividido. Un valor alto es útil para evitar el overfitting, ya que previene divisiones demasiado específicas.

Justificación del rango: Un límite superior de 50 es más que suficiente, ya que solo contamos con 500 observaciones en total, y para el Cross-Validation solo usamos el 90 % dividido en 5 folds (lo que deja unos 360 puntos por fold de entrenamiento).

- `min_weight_fraction_leaf`: `uniform(loc=0, scale=0.5)`

Es la fracción mínima de las muestras totales necesaria para formar un nodo hoja. Este parámetro es otro mecanismo para regular el overfitting, ya que evita que existan hojas basadas en muy pocas muestras.

Justificación del rango: Abarcamos el rango completo de valores posibles, de 0 a 1.

- `criterion`: `('gini', 'entropy', 'log_loss')`

Es la medida utilizada para evaluar la ganancia de información al realizar una división.

Justificación del rango: Exploramos las tres opciones disponibles.

- **Hiperparametros no considerados**: `-class_weight`

Inicialmente pensamos que sería clave por la diferencia de proporción en el target, pero al realizar la cross-validation, la performance fue mejor en su configuración default.

Resultados: Tras 200 iteraciones, la mayor performance medida con Cross-Validation AUCROC fue de 0.709 y se logró con:

```
'criterion': 'entropy',  
'max_depth': 4,  
'min_samples_split': 18,  
'min_weight_fraction_leaf': 0.067
```

3.1.2. KNN

Los parámetros que exploramos fueron:

- **n_neighbors:** np.arange(2, 50)

Define el número (K) de vecinos más cercanos que se consideran para clasificar una nueva muestra. Un valor pequeño causa que el modelo se ajuste más a los datos de entrenamiento (potencial overfitting), mientras que un valor grande “suaviza” la decisión, clasificando de forma más general (potencial underfitting).

Justificación del rango: Dado que nuestro dataset de entrenamiento tiene aproximadamente 450 observaciones (90 % de 500) que se dividen en 5 folds (lo que deja unos 360 puntos por fold de entrenamiento), el rango de exploración de hasta 50 vecinos es más que suficiente.

- **weights:** ['uniform', 'distance']

Define cómo se pondera la contribución de cada vecino en la clasificación de una nueva muestra. Consideramos que la opción 'distance' podría mejorar la performance, ponderando más los vecinos cercanos.

Consideramos este un hiperparámetro clave, y esperábamos que la performance mejorara con la opción 'distance'.

Justificación del rango: Probamos ambas opciones disponibles

- **p:** [1,2,3]

Determina el tipo de distancia utilizada para encontrar a los vecinos más cercanos.

Justificación del rango: Incluimos p=1 (Manhattan), p=2 (Euclídea) y p=3 (distancia de orden superior) para evaluar posibles mejoras.

- **Hiperparametros no considerados:** -algorithm -njobs

No los consideramos relevantes ya que tienen que ver con mejorar la eficiencia del cómputo durante el entrenamiento, y no con la performance del modelo en sí.

Resultados: Tras 50 iteraciones, la mayor performance medida con Cross-Validation AU-CROC fue de 0.869 con:

```
'n_neighbors': 14,  
'weights': 'distance',  
'p': 1
```

3.1.3. SVM

Los parámetros que exploramos fueron:

- **C:** uniform(0,100)

Controla el trade-off entre margen amplio y clasificación correcta.

Justificación del rango: Empezamos probando un rango amplio y continuo de 0 a 100 para C. Este rango nos pareció razonable para cubrir tanto valores pequeños (que permiten más errores) como valores grandes (que exigen mayor precisión), asegurando que encontráramos el punto óptimo de regularización.

- `class_weight`: [None,'balanced']

Ajusta la importancia de la clase minoritaria. Resultó clave debido al desbalance de datos.

Al igual que en el Árbol de Decisión, pensamos que este hiperparámetro sería crucial dado el desbalance de nuestros datos. En el caso de SVM, este parámetro sí marcó la diferencia.

Justificación del rango: Probamos ambas opciones disponibles

- **Hiperparametros no considerados**: -Kernel

También probamos diferentes valores para el hiperparámetro kernel (que define la función utilizada para mapear los datos a un espacio de dimensiones más altas) junto con sus parámetros dependientes (gamma y degree). Sin embargo, observamos que la mejor performance se obtenía con el valor por defecto: rbf (Radial Basis Function).

Resultados: Tras 50 iteraciones, la mayor performance medida con Cross-Validation AUCROC fue de 0.897 con:

```
'C': 5.8083612168199465,
'class_weight': 'balanced'
```

3.2.

3.2.1. LDA y Naive Bayes

Con LDA se obtuvo un Cross-Validation AUCROC de 0.688 mientras que con Naive Bayes fue 0.7845. Ambos algoritmos parten de la suposición de que los atributos siguen una distribución Gaussiana (normal). Sin embargo, su distinción fundamental radica en la forma en que gestionan la relación entre los atributos:

- Naive Bayes asume la independencia condicional de todos los atributos dada la clase (es decir, ignora la covarianza entre ellos).
- LDA modela la distribución de los datos calculando y utilizando la matriz de covarianza de los atributos, bajo el supuesto de que esta matriz es idéntica para todas las clases.

Dado que en este caso el dataset cuenta con 200 atributos, creemos que el cálculo y uso de la matriz de covarianza por parte de LDA (por la alta dimensionalidad) podría haber resultado en una desventaja que se reflejó en su performance frente a la simplicidad de Naive Bayes.

Hiperparámetros de Optimización Clave:

Un hiperparámetro que podría ser importante para la mejora de la performance en ambos algoritmos es el ajuste de las prioridades de clase (priors). Dado que nuestro dataset está desbalanceado (aproximadamente 30 % clase positiva y 70 % negativa), por defecto ambos algoritmos estiman las priors como la simple proporción observada de clases. Esto resulta en una probabilidad calculada que favorece a la clase mayoritaria (en este caso negativa), incrementando el riesgo de que el modelo clasifique incorrectamente instancias positivas como negativas, y por ende generando un número elevado de Falsos Negativos (FN).

Específicamente para LDA, el hiperparámetro *shrinkage* podría ser clave, ya que aplica un factor de encogimiento a la matriz de covarianza que podría ser particularmente útil cuando el número de muestras es pequeño en comparación con la cantidad de atributos (nuestro caso).

3.3.

Al evaluar los resultados, observamos que SVM fue el algoritmo con el mayor AUCROC obtenido. Creemos que el buen desempeño de SVM era esperable por varias razones clave:

- **Robustez:** No hace suposiciones tan fuertes sobre la distribución de los datos. Además, solo es sensible a los puntos cercanos al margen (vectores de soporte), disminuyendo sensibilidad a variaciones entre el set de entrenamiento y el de prueba.
- **Manejo de la Alta Dimensionalidad:** SVM es capaz de capturar fronteras complejas no lineales y en espacios de alta dimensionalidad gracias al truco del *kernel*, que le permite encontrar un hiperplano de separación en espacios de altas dimensiones sin el costo computacional de tener que hacer cálculos explícitos en ese espacio. En el caso de este dataset que contiene una gran cantidad de columnas, creemos que esta característica es clave.
- **Regulación del Overfitting:** El modelo maneja bastante bien el overfitting, principalmente al regular el hiperparámetro C .

Respecto al resto de los modelos, sus resultados se alinearon con nuestras expectativas:

El modelo K-Nearest Neighbors (KNN) nos dio el segundo mejor *score*. Pensamos que no superó a SVM principalmente por el desbalance de clases, que puede generar Falsos Negativos (FN), y porque la alta dimensionalidad podría complicar la elección de vecinos representativos ya que todos los atributos contribuyen por igual a la distancia. Por su parte, creemos que el Árbol de Decisión mostró tendencia al overfitting, por la gran sensibilidad de su performance a los cambios en el parámetro *max_depth*.

El modelo de LDA tuvo un mal desempeño, lo cual atribuimos a su asunción de normalidad en los datos y a que solo genera fronteras lineales, cuando en este caso suponemos que las fronteras de los datos son más complejas dada la alta dimensionalidad del dataset.

Finalmente, Naive Bayes Gaussiano nos sorprendió: a pesar de que asume normalidad e independencia entre atributos, obtuvo un buen resultado de AUC-ROC, superando a LDA y a los Árboles. Este resultado se alinea con lo que dice la bibliografía, la cual dice que este modelo suele funcionar muy bien cuando hay pocos datos en comparación con la cantidad de atributos.

4. Evaluación de performance

Para este ejercicio, usamos el mejor modelo encontrado, el SVM con la configuración hallada en *RandomizedSearchCV*, reentrenado sobre todo el conjunto de entrenamiento con los mejores hiperparámetros. Para estimar su desempeño real decidimos usar el dataset de control (nunca visto por el modelo). Calculamos el AUCROC y obtuvimos un valor de 0.9776, sorprendentemente alto, casi un 9 % más que el promedio de validación cruzada (0.8969). Para entender mejor esta diferencia, revisamos los resultados *fold* a *fold* de la validación: el mejor *fold* había alcanzado un AUCROC de 0.928, todavía por debajo del 0.9776 obtenido con el control.

Sin embargo, creemos que esto se debe a que el dataset de control era muy chico (solo 50 muestras), mientras que cada *fold* de validación usaba unas 90. Al ser tan pocas instancias, es posible que el conjunto de control justo incluyera casos más fáciles de clasificar. Por eso creemos que la estimación de *performance* no es confiable. En un trabajo futuro, sería más adecuado guardar un conjunto de control considerablemente más grande si estamos buscando una estimación precisa del desempeño del modelo.

Además, repetimos el cálculo de AUCROC para KNN y árbol de decisión sobre el mismo conjunto de control y en ambos casos pasó lo mismo: los *scores* fueron más altos que los promedios de validación cruzada. Esto refuerza la idea de que este dataset en particular tenía instancias más “fáciles” de clasificar. Las curvas ROC de SVM, KNN y árbol se muestran en la Figura 1. Allí también se confirma que SVM fue el modelo con mejor desempeño, seguido de KNN y luego el árbol.

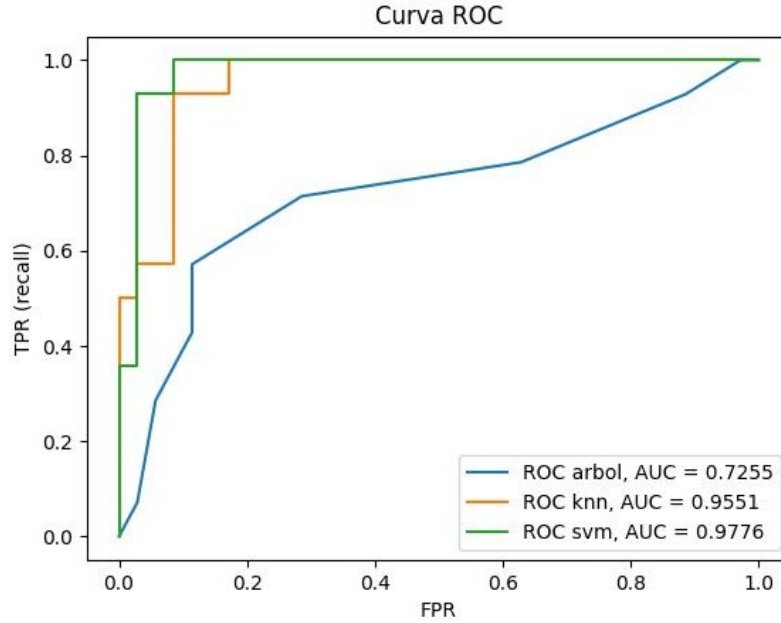


Figura 1: Curvas ROC de los tres modelos, con el AUROC asociado sobre el conjunto de control. Por otro lado, se mantiene el orden de los scores.

5. Conclusiones

En este trabajo construimos y comparamos distintos modelos de aprendizaje automático para un problema de clasificación binaria sobre un dataset de 500 muestras con mediciones de ARN de 200 genes, extraídas de pacientes con lesiones pre-tumorales.

En el Ejercicio 1, separamos los datos en un conjunto de desarrollo (90 %) y uno de control (10 %), cuidando que la proporción de clases se mantuviera en ambos conjuntos, ya que el dataset estaba desbalanceado y contábamos con pocos datos.

En el Ejercicio 2, entrenamos un árbol de decisión con altura máxima 3 y evaluamos su *performance* con validación cruzada ($K=5$). Calculamos métricas de *Accuracy*, AUCROC y AUPRC tanto en entrenamiento como en validación, además del *score* global. Vimos que la *Accuracy* tendía a ser más alta, pero poco confiable en datasets desbalanceados. En cambio, el AUCROC resultó más adecuado para evaluar el problema. También probamos distintas combinaciones de hiperparámetros (criterio de corte y altura máxima) y observamos que, a mayor profundidad del árbol el modelo tendía a sobreajustar.

En el Ejercicio 3, estudiamos el desempeño de árboles, KNN y SVM usando *Randomized-SearchCV* para distintas configuraciones de hiperparámetros. Además, comparamos con los modelos LDA y Naïve Bayes. El modelo con mejor desempeño fue SVM, que logró el AUCROC más alto. Esto era esperable, ya que los modelos SVM suelen rendir bien en datasets de alta dimensionalidad y con pocas instancias porque hacen uso de unos pocos vectores de soporte.

En el Ejercicio 4, calculamos las probabilidades (de la clase positiva) que devuelve nuestro mejor modelo (SVM) de las distintas instancias de un dataset $X_{\text{held_out}}$ y estimamos la *performance* de dicho modelo con el conjunto de control. Obtuvimos un AUCROC de 0.9776, considerablemente mayor que el promedio de *cross-validation* (0.8969). Creemos que esta diferencia se debe a que el conjunto de control era muy pequeño (49 muestras) y que esas instancias, por casualidad, resultaron más sencillas de clasificar correctamente. Concluimos que con tan pocas instancias no se puede estimar de forma robusta la *performance* real del modelo.

En conclusión, el trabajo nos permitió aprender acerca de la importancia de la separación correcta de los datos, las diferencias entre métricas para datasets desbalanceados y el efecto de los distintos hiperparámetros. Para próximos trabajos, contar con un conjunto de test más grande, permitiría estimar con mayor precisión la verdadera capacidad de generalización del modelo.